Global Payouts — Dependency Map

Living reference · Companion to the Functional Spec glossary. **The Stripe API is global for acceptance, but a Stripe account is a single Merchant of Record.** This map tracks, per market, who is paid, in what currency it settles, and the dependency that gates going local.

Status: **Living reference · Globalized Commerce settlement layer**

Scope: **How an agentic purchase gets from *authorized* to *money-in-a-bank*, per market, and what each market depends on.**

> Companion to `FUNCTIONAL-SPEC.md` §"Glossary — Globalized Commerce". This document is the **payouts dependency map**: who is the Merchant of Record per market, in what currency it settles, and the legal/contract blocker that gates going local.

---

1. The core principle

The Stripe API is global for *acceptance*, but a Stripe account is a *single* Merchant of Record (MoR).

One Stripe account can charge a card (or a Google Pay / wallet token) from a shopper **anywhere**, but it **settles in its home country's currency to a home-country bank**. "Going global" therefore splits into two independent questions:

1. **Acceptance** — can we charge this shopper's instrument? (Almost always yes, via the wallet → Stripe.)

2. **Payout / MoR** — which legal entity receives the money, in what currency, to which bank? (Singular per Stripe account.)

The settlement layer models this explicitly so the truth is visible per market rather than assumed.

---

2. The dependency map

Reference account: `yslnew` — a **US** Stripe account (USD, payouts to a US bank, `charges_enabled`). Every non-US sale is therefore **cross-border under the US entity** until a per-region entity exists.

MARKET	MoR ENTITY (who is paid)	ACQUIRER	PAYOUT → BANK	DEPENDENCY / BL
US	yslnew (US) — LIVE	Stripe US	USD → US bank	✓ none (clear p
CA/GB/EU	yslnew (US) — x-border	Stripe US	USD → US (FX)	local-currency
JP	yslnew (US) — x-border	Stripe US	USD → US (FX)	JPY local + Kc
AE	yslnew (US) — x-border	Stripe US	USD → US (FX)	local needs a
KR	Platform KR (Korean Biz ID)	Kakao Pay	KRW → KR bank	Kakao 가맹점 cor
CN / TW	Platform local entity	domestic PG	local → local bank	local entity +
KP/IR/SY	—	—	—	sanctioned — r

The three columns that drive everything: **(1) the legal entity that is MoR**, **(2) local-currency payout vs USD cross-border**, **(3) the gating contract/entity**.

---

3. MoR classes

The Payments operator screen (`/embed/payments` → **Rails** tab) labels every market with one of:

MOR CLASS	MEANING	MARKETS TODAY
local	Settled by a Stripe entity **in that country**, local-currency payout	**US**
x-border	Accepted under the **US** entity, settled **USD** (FX) — works now, no local entity	**JP, DE, FR, CA, GB, AU, SG, AE**
domestic	Off-Stripe — a **domestic acquirer** (Kakao / PG) under a local entity	**KR, CN, TW**
blocked	Sanctioned — never settable	**KP, IR, SY**

`/commerce/markets` stamps this per market (`mor`), derived from the Stripe home country + the market registry's acquirer. It flips from **x-border → local** automatically the moment a regional Stripe entity is registered (acquirer `stripe_xborder → stripe_native`).

4. Per-region paths

- **North America (US/CA).** US is home (LIVE). CA accepted cross-border now; add a CA Stripe entity only if local CAD payout is required.
- **EU (DE/FR/…).** Accepted cross-border under the US entity today (Google Pay `CRYPTOGRAM_3DS` satisfies SCA/3DS). For **EUR** local settlement + EU methods, register an **EU Stripe entity**.
- **Japan.** Accepted cross-border now. For **JPY** local settlement, native methods (Konbini, PayPay, JCB), and a JP bank payout, register a **Stripe KK (Japanese entity)**. The Korean Business ID does **not** unlock Stripe Japan.

- **Korea.** Stripe **cannot** be MoR in KR. Settles **domestically via Kakao Pay** under the platform's **Korean Business ID** (single merchant of record; vendors are sub-merchant tenants). Blockers: Kakao 가맹점 (merchant) contract, 전자금융거래법 (PG/PSP) compliance, escrow (구매안전서비스).
- **Rest of Asia (CN/TW/…).** Domestic PG + local entity. Incubating in the registry until ratified.
- **Sanctioned (KP/IR/SY).** Hard-blocked at the guardrail; never resolvable.

---

5. How the architecture encodes the map

- **Market registry** (`MARKET_MODULES`) — per-market `{ rails, acquirer, settle_currency, lifecycle }`. Two-tier: **ratified** (live: JP/KR) vs **incubating/blocked**.
- **Settlement sockets** — pluggable per-Country/Bank connectors (Stripe / Kakao / domestic PG), each carrying its MoR class. Adding a region = **plugging in a socket**, not a rewrite.
- **Fail-closed guardrail** — `assertMarketActive` (runtime) + a pre-deploy guard refuse any non-ratified market reaching active settlement dispatch. A market is only ratifiable once its socket exists.
- **Adding a region is a config swap:** register the regional Stripe entity → set the market's acquirer to `stripe_native` → its MoR flips to **local** and payout becomes local-currency.

---

6. Cross-cutting dependencies (every market)

These gate **all** payouts regardless of region:

- **Identity — OAuth / Xano Auth Me.** Every purchase is bound to the signed-in shopper (`crm_jwt`) via `bindCredentialToAuthMe` before any capture; the wallet token is only the instrument.
- **Authorization — AP2 mandate.** A signed mandate (cap / scope / rail) must authorize the agent; enforced by `checkMandateLimits`.
- **System of record — Shopify.** The sale lands as a Shopify draft → paid → real Order (keeps the Shopify + GA4 data benefit). Shopify is the *sell*; settlement is external.
- **Machine legibility — GA4.** Every outcome emits a labeled server-side event: `ucp_checkout_created`, `ucp_agent_purchase`, `ucp_checkout_blocked` — readable by agents/analytics without a UI.

---

7. Wallet rails (the instrument layer)

Wallets are **acceptance**, not MoR — they tokenize and settle via the PSP socket:

WALLET	STATUS	SETTLES VIA	SURFACE
Google Pay	**wired live** (needs keys + Google Wallet Console)	Stripe socket (gateway token)	prominent — nests into OAuth
Apple Pay	scaffold	Stripe socket (PKPaymentToken)	prominent — iOS checkout
Samsung Pay	scaffold	Stripe x-border / domestic PG	secondary — `/payments`
Kakao Pay	live	Kakao (domestic, KR)	KR
Cash App / Venmo	live (manual)	wallet deep-link	cross-platform

A Google Pay purchase in any market settles through whatever MoR that market resolves to in the map above.

---

8. Current status

- ✓ **US** — live-capable on the US Stripe account (TEST first; clear `proof_of_liveness`).
- ✓ **Cross-border acceptance** — JP/EU/CA/GB/AU/SG/AE under the US entity (USD payout).
- ✓ **Korea** — Kakao Pay live (domestic).
- **Local-currency payout** for JP/EU/AE — pending a regional Stripe entity (a config swap when ready).
- **CN/TW + other domestic markets** — pending local entity + PG contracts.
- **Sanctioned** — permanently blocked.

The dependency that turns most / rows green is the same one each time: **a legal entity in that market** (a regional Stripe account, or a local merchant registration + domestic PG contract). The code is already shaped to flip on that.
